0.1 Building Pairing-based SNARK

0.1.1 Attempt #1: Encrypted Verification

Now, assume we have the cyclic group $\mathbb{G}$ of prime order q with a generator g. Typically, this is the group of points on an elliptic curve. Assume for simplicity that $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$ is a symmetric pairing function, where $(\mathbb{G}_T, \times)$ is a target group.

Now, suppose during the setup phase, we have a trusted party that generated a random value $\tau \xleftarrow{R} \mathbb{F}$ and public parameters $g^\tau, g^{\tau^2}, \ldots, g^{\tau^d}$ for $d = 2|C|$ — maximum degree of used polynomials (later we will use notation $\{g^{\tau^i}\}_{i\in[d]}$ for brevity). Then, party **deleted** τ. This way, we can now find the KZG commitment for each polynomial. Indeed, for example,

$$\mathsf{com}(L) \triangleq g^{L(\tau)} = g^{\sum_{i=0}^{d} L_i \tau^i} = \prod_{i=0}^{d} (g^{\tau^i})^{L_i},$$

and the same goes for $g^{R(\tau)}, g^{O(\tau)}, g^{H(\tau)}, g^{Z(\tau)}$. Now, with these points, how can we verify that the polynomial $M(x) = L(x)R(x) - O(x)$ is correct? Well, first notice that the check is equivalent to

$$L(\tau)R(\tau) = Z(\tau)H(\tau) + O(\tau).$$

Notice that we transferred $O(\tau)$ to the right side of the equation to further avoid finding the inverse. Now, we can check this equality using encrypted values as follows:

$$e(\mathsf{com}(L), \mathsf{com}(R)) = e(\mathsf{com}(Z), \mathsf{com}(H)) \cdot e(\mathsf{com}(O), g),$$

Remark. One might ask: why is the above equation correct? Well, let us see:

$$\begin{aligned}
& e(\mathsf{com}(L), \mathsf{com}(R)) = e(\mathsf{com}(Z), \mathsf{com}(H)) \cdot e(\mathsf{com}(O), g) && \text{// Initial statement} \\
\Leftrightarrow\ & e(g^{L(\tau)}, g^{R(\tau)}) = e(g^{Z(\tau)}, g^{H(\tau)}) \cdot e(g^{O(\tau)}, g) && \text{// KZG commitment def.} \\
\Leftrightarrow\ & e(g, g)^{L(\tau)R(\tau)} = e(g, g)^{Z(\tau)H(\tau)} e(g, g)^{O(\tau)} && \text{// Pairing bilinearity} \\
\Leftrightarrow\ & e(g, g)^{L(\tau)R(\tau)} = e(g, g)^{Z(\tau)H(\tau)+O(\tau)} && \text{// Exponent product rule} \\
\Leftrightarrow\ & L(\tau)R(\tau) = Z(\tau)H(\tau) + O(\tau) && \text{// QAP Check} \\
\Leftrightarrow\ & L(x)R(x) \equiv Z(x)H(x) + O(x) && \text{// Schwarz-Zippel Lemma}
\end{aligned}$$

So, sounds like we are done. Let us summarize what we have done so far:

Attempt #1: Non-sound SNARK Protocol

Suppose we are given a circuit C with a maximum degree d of polynomials used underneath. Thus, all parties additionally know the target polynomial $Z(x)$.

Setup(1^λ)

The *trusted party* conducts the following steps:

- ✓ Picks a random value $\tau \xleftarrow{R} \mathbb{F}$.
- ✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$.
- ✓ **Deletes** τ (toxic waste).
- ✓ **Outputs** prover parameters $\mathsf{pp} \leftarrow \{g^{\tau^i}\}_{i\in[d]}$ and verifier parameters $\mathsf{vp} \leftarrow \mathsf{com}(Z)$.

Prove($\mathsf{pp}, \mathbf{x}, \mathbf{w}$)

The prover $\mathcal{P}$ conducts the following steps:

- ✓ Runs the circuit with the statement $\mathbf{x}$ and witness $\mathbf{w}$, obtains the intermediate constraint values, and calculates the polynomials $L(x)$, $R(x)$, $O(x)$ through Lagrange Interpolation.
- ✓ Calculates $H(x) \leftarrow (L(x)R(x) - O(x))/Z(x)$.
- ✓ Calculates the KZG commitments as follows:

$$\pi_L \leftarrow \mathsf{com}(L), \pi_R \leftarrow \mathsf{com}(R), \pi_O \leftarrow \mathsf{com}(O), \pi_H \leftarrow \mathsf{com}(H),$$

  using powers of τ from the prover parameters pp.
- ✓ Publishes $\boldsymbol{\pi} = (\pi_L, \pi_R, \pi_O, \pi_H)$ as a proof.

Verify($\mathsf{vp}, \mathbf{x}, \boldsymbol{\pi}$)

Upon receiving $\boldsymbol{\pi} = (\pi_L, \pi_R, \pi_O, \pi_H)$, using $\mathsf{com}(Z)$ from the verifier parameters vp, the verifier $\mathcal{V}$ checks:

$$e(\pi_L, \pi_R) = e(\mathsf{com}(Z), \pi_H) \cdot e(\pi_O, g).$$

This sounds like an end to the story. However, there is a problem with this approach: there is no guarantee that commitments π_L, π_R, π_O, π_H were indeed obtained through exponentiating the base g by the corresponding values $L(\tau)$, $R(\tau)$, $O(\tau)$, and $H(\tau)$. In other words, how can the verifier know that prover indeed knows, say, polynomial $L(x)$ such that $\pi_L = g^{L(\tau)}$?

0.1.2 Attempt #2: Including Proof of Exponent

In this section, we introduce the **Proof of Exponent assumption** (PoE) which makes KZG knowledge sound. Let us define it below.

Definition 0.1.1 (Proof of Exponent for KZG Commitment)**.** A **Proof of Exponent** (PoE) is a protocol that allows the prover $\mathcal{P}$ to convince the verifier $\mathcal{V}$ that he obtained a value $\mathsf{com}(f)$ through exponentiating a base g by $f(\tau)$. The protocol works as follows:

1. **Setup:** Proper parameters pp now contain not only $\{g^{\tau^i}\}_{i\in[d]}$, but also $\{g^{\alpha\tau^i}\}_{i\in[d]}$ for randomly selected $\tau, \alpha \xleftarrow{R} \mathbb{F}$ and further **deleted** values.
2. **Commit:** $\mathcal{P}$ commits to two values: $\mathsf{com}(f) = g^{f(\tau)}$ and $\mathsf{com}'(f) = g^{\alpha f(\tau)}$. The latter can be computed using pp as follows:

$$\mathsf{com}'(f) = \prod_{i=0}^{d} (g^{\alpha\tau^i})^{f_i}$$

3. **Verify:** $\mathcal{V}$ additionally checks $e(\mathsf{com}(f), g^\alpha) = e(\mathsf{com}'(f), g)$.

The informal reason why it makes KZG commitment sound is following: suppose we have an adversary prover $\mathcal{P}^*$ that published commitment c without knowing underlying polynomial $f(x)$. Now, there is no way for him to cheat the verifier. Indeed, what $\mathcal{P}^*$ needs to do is calculating c^α, but he does now know α since, similarly to τ, it was deleted as a part of the toxic waste. Besides, he cannot obtain α for the same reason he cannot obtain τ.

For that reason, we modify the SNARK protocol to include not only commitments to polynomials but also PoE for each of them. Let us see how it looks like.

Attempt #2: SNARK with PoE included

Suppose we are given a circuit C with a maximum degree d of polynomials used underneath. Thus, all parties additionally know the target polynomial $Z(x)$.

Setup(1^λ)

The *trusted party* conducts the following steps:

- ✓ Picks a random value $\tau, \alpha \xleftarrow{R} \mathbb{F}$.
- ✓ Calculates the public parameters $\{g^{\tau^i}\}_{i\in[d]}$, $\{g^{\alpha\tau^i}\}_{i\in[d]}$.
- ✓ **Deletes** τ, α (toxic waste).
- ✓ **Outputs** proper parameters $\mathsf{pp} \leftarrow \{\{g^{\tau^i}\}_{i\in[d]}, \{g^{\alpha\tau^i}\}_{i\in[d]}\}$, and verification parameters $\mathsf{vp} \leftarrow \{g^{Z(\tau)}, g^\alpha\}$.

Prove($\mathsf{pp}, \mathbf{x}, \mathbf{w}$)

The prover $\mathcal{P}$ conducts the following steps:

- ✓ Runs the circuit with the statement $\mathbf{x}$ and witness $\mathbf{w}$, obtains the intermediate constraint values, and calculates the polynomials $L(x), R(x), O(x)$ through Lagrange Interpolation.
- ✓ Calculates $H(x) \leftarrow (L(x)R(x) - O(x))/Z(x)$.
- ✓ Calculates the sound KZG commitments as follows:

$$\begin{aligned} \pi_L &\leftarrow g^{L(\tau)}, & \pi'_L &\leftarrow g^{\alpha L(\tau)} \\ \pi_R &\leftarrow g^{R(\tau)}, & \pi'_R &\leftarrow g^{\alpha R(\tau)} \\ \pi_O &\leftarrow g^{O(\tau)}, & \pi'_O &\leftarrow g^{\alpha O(\tau)} \\ \pi_H &\leftarrow g^{H(\tau)}, & \pi'_H &\leftarrow g^{\alpha H(\tau)}. \end{aligned}$$

  using powers $\{g^{\tau^i}\}_{i\in[d]}$ and $\{g^{\alpha\tau^i}\}_{i\in[d]}$ from the proper parameters pp.
- ✓ Publishes $\boldsymbol{\pi} = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi'_H)$ as a proof.

Verify($\mathsf{vp}, \mathbf{x}, \boldsymbol{\pi}$)

Upon receiving $\boldsymbol{\pi} = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi'_H)$, the verifier $\mathcal{V}$ checks:

$$\begin{aligned} e(\pi_L, \pi_R) &= e(g^{Z(\tau)}, \pi_H) \cdot e(\pi_O, g), & &\textit{// Polynomial Equality Test} \\ e(\pi_L, g^\alpha) = e(\pi'_L, g), \quad e(\pi_R, g^\alpha) &= e(\pi'_R, g), & &\textit{// Proof of Exponent} \\ e(\pi_O, g^\alpha) = e(\pi'_O, g), \quad e(\pi_H, g^\alpha) &= e(\pi'_H, g). & &\textit{// Proof of Exponent} \end{aligned}$$

The provided protocol is secure under the PoE assumption. However, it is still not fully sound. Currently, there is no guarantee that when evaluating π_L, π_R, π_O we used the same extended witness $\mathbf{w}$. In other words, the prover can still cheat by using different extended witnesses for each polynomial (faking the proof system is still hard in this situation, but we want to make sure to eliminate all possible weaknesses). Let us see how to fix this!

0.1.3 Attempt #3: Making SNARK Sound

Besides fixing the issue with consistent use of witness $\mathbf{w}$, we additionally include one more optimization we have not included so far.

Optimization. Left/Right/Output Polynomial Preprocessing. Recall that:

$$L(x) = \sum_{i=0}^{n} w_i L_i(x), \quad R(x) = \sum_{i=0}^{n} w_i R_i(x), \quad O(x) = \sum_{i=0}^{n} w_i O_i(x).$$

while $M(x) = L(x)R(x) - O(x)$ is only known to the prover $\mathcal{P}$ since it contains the extended witness ($\mathbf{w}$) coefficients. However, the set of polynomials

$$\{L_i(x)\}_{i\in[n]}, \{R_i(x)\}_{i\in[n]}, \{O_i(x)\}_{i\in[n]}$$

are known in advance. Meaning, we can precompute the values of $\{g^{L_i(\tau)}\}_{i\in[n]}$, $\{g^{\alpha L_i(\tau)}\}_{i\in[n]}$, $\{g^{R_i(\tau)}\}_{i\in[n]}$, $\{g^{\alpha R_i(\tau)}\}_{i\in[n]}$, $\{g^{O_i(\tau)}\}_{i\in[n]}$, $\{g^{\alpha O_i(\tau)}\}_{i\in[n]}$ and use them in the prover parameters pp.

How? Suppose the prover $\mathcal{P}$ knows the extended witness $\mathbf{w}$. Consider the polynomial $L(x) = \sum_{i=0}^{n} w_i L_i(x)$. $\mathcal{P}$ can compute the KZG commitment π_L and its PoE π'_L as follows:

$$\pi_L \triangleq g^{L(\tau)} = g^{\sum_{i=0}^{n} w_i L_i(\tau)} = \prod_{i=0}^{n} (g^{L_i(\tau)})^{w_i}, \quad \pi'_L \triangleq g^{\alpha L(\tau)} = g^{\alpha \sum_{i=0}^{n} w_i L_i(\tau)} = \prod_{i=0}^{n} (g^{\alpha L_i(\tau)})^{w_i}.$$

Fix. Witness consistency check.

Introducing new term with β. To prove that the same $\mathbf{w}$ is used in all commitments, we need some "checksum" term that will somehow combine all polynomials $L(x)$, $R(x)$, and $O(x)$ with the witness $\mathbf{w}$. Moreover, we will need to compare this term with proofs π_L, π_R, and π_O. The best candidate for this is the following group element for some other random $\beta \xleftarrow{R} \mathbb{F}$:

$$\pi_\beta = g^{L(\tau)+R(\tau)+O(\tau)} = \prod_{i=1}^{n} (g^{L_i(\tau)+R_i(\tau)+O_i(\tau)})^{w_i}, \ \pi'_\beta = \prod_{i=1}^{n} (g^{\beta(L_i(\tau)+R_i(\tau)+O_i(\tau))})^{w_i}$$

This way, we get a term that includes all three polynomials $L(x)$, $R(x)$, and $O(x)$, and all coefficients of the extended witness $\mathbf{w}$. Moreover, verifier $\mathcal{V}$ can compare π_β with three other commitments π_L, π_R, π_O to ensure that all of them are consistent. This is done through the following check:

$$e(\pi_L \pi_R \pi_O, g^\beta) = e(\pi'_\beta, g).$$

Again, this approach still has a weakness (yeah-yeah, I am also tired of this). Indeed, this check is complete (meaning, if $\mathbf{w}$ is used consistently across π_L, π_R, π_O, then the check will pass), but it is still not sound with an overwhelming probability. Indeed, suppose $\mathbf{w}$ is used inconsistently, meaning, we have extended witnesses $\mathbf{w}_L$, $\mathbf{w}_R$, $\mathbf{w}_O$, and $\mathbf{w}_\beta$, each for corresponding polynomials. If the witness is consistent, the following condition must hold:

$$(w_{L,i} L_i(\tau) + w_{R,i} R_i(\tau) + w_{O,i} O_i(\tau))\beta = w_{\beta,i}\beta(L_i(\tau) + R_i(\tau) + O_i(\tau)) \quad \forall i \in [n]$$

Assume otherwise. Consider a simple situation where it happens that $L_i \equiv R_i$, meaning $L_i(\tau)$ and $R_i(\tau)$ are the same (call them q). Then,

$$(w_{L,i} + w_{R,i})q + w_{O,i} O_i(\tau) = w_{\beta,i}(2q + O_i(\tau)) \quad \forall i \in [n]$$

For arbitrarily chosen $w_{R,i}$ and $w_{O,i}$, the adversary prover $\mathcal{P}^*$ can set $w_{\beta,i} := w_{O,i}$ and $w_{L,i} = 2w_{O,i} - w_{R,i}$. It is easy to verify that the above equation would hold, meaning that $\mathbf{w}$ is not the same across all polynomials.

One might ask: well, situation when $L_i \equiv R_i$ is very rare! Indeed, but there also might be situations where $L_i \equiv 5R_i$, $R_i \equiv 100O_i$, or $L_i \equiv 235O_i$ — all of them would lead to the same issue. So what is the solution?

Introducing separate β_L, β_R, β_O. Our proposal is to make β different for each polynomial $L(x)$, $R(x)$, $O(x)$, making it much harder for the adversary to find inconsistent witnesses. That being said, during the setup phase, we choose arbitrary $\beta_L, \beta_R, \beta_O \xleftarrow{R} \mathbb{F}$ and publish $\{g^{\beta_L L_i(\tau)+\beta_R R_i(\tau)+\gamma_O O_i(\tau)}\}_{i\in[n]}$ as a part of the prover parameters pp. Then, the prover $\mathcal{P}$ calculates the following additional commitment:

$$\pi_\beta \leftarrow \prod_{i=1}^{n} (g^{\beta_L L_i(\tau)+\beta_R R_i(\tau)+\beta_O O_i(\tau)})^{w_i}$$

and then publishes π_β as a part of the proof. The verifier $\mathcal{V}$ checks the following condition:

$$e(\pi_L, g^{\beta_L}) \cdot e(\pi_R, g^{\beta_R}) \cdot e(\pi_O, g^{\beta_O}) = e(\pi_\beta, g).$$

Even that is not the end of the story! We also need to make sure that g^{β_L}, g^{β_R}, and g^{β_O} are incompatible with $g^{\sum_{i=0}^{n}(\beta_L L(\tau)+\beta_R R(\tau)+\beta_O O(\tau))w_i}$ (for reasons we drop here due to already long explanation). For that reason, we multiply β_L, β_R, β_O by some random factor $\gamma \xleftarrow{R} \mathbb{F}$. This way, the proving part remains the same, but the check becomes:

$$e(\pi_L, g^{\gamma\beta_L}) \cdot e(\pi_R, g^{\gamma\beta_R}) \cdot e(\pi_O, g^{\gamma\beta_O}) = e(\pi_\beta, g^{\gamma}).$$

Oof, that was a long fix! Let us come back to the new version of the SNARK protocol (not yet zero-knowledge)!

Attempt #3: Sound SNARK Protocol

Suppose we are given a circuit C with a maximum degree d of polynomials used underneath. Thus, all parties additionally know the target polynomial $Z(x)$.

Setup(1^λ)

The *trusted party* conducts the following steps:

- ✓ Picks random values $\tau, \alpha, \beta_L, \beta_R, \beta_O, \gamma \xleftarrow{R} \mathbb{F}$.
- ✓ **Outputs** prover parameters pp and verification parameters vp:

$$\begin{aligned}\mathsf{pp} \leftarrow \Big\{ &\{g^{\tau^i}\}_{i\in[d]}, \{g^{L_i(\tau)}, g^{\alpha L_i(\tau)}, g^{R_i(\tau)}, g^{\alpha R_i(\tau)}, \\ &g^{O_i(\tau)}, g^{\alpha O_i(\tau)}, g^{\beta_L L_i(\tau)+\beta_R R_i(\tau)+\beta_O O_i(\tau)}\}_{i\in[n]}\Big\} \\ \mathsf{vp} \leftarrow \Big\{ &g^{Z(\tau)}, g^{\alpha}, g^{\beta_L}, g^{\beta_R}, g^{\beta_O}, g^{\beta_L\gamma}, g^{\beta_R\gamma}, g^{\beta_O\gamma}, g^{\gamma}\Big\}\end{aligned}$$

- ✓ **Deletes** aforementioned random scalars (toxic waste).

Prove(pp, $\mathbf{x}$, $\mathbf{w}$)

The prover $\mathcal{P}$ conducts the following steps:

- ✓ Runs the circuit to get $\mathbf{w}$ and $L(x), R(x), O(x)$.
- ✓ Calculates $H(x) \leftarrow (L(x)R(x) - O(x))/Z(x)$.
- ✓ Calculates the sound KZG commitments as follows:

$$\begin{aligned}\pi_L &\leftarrow g^{L(\tau)}, & \pi'_L &\leftarrow g^{\alpha L(\tau)}, \\ \pi_R &\leftarrow g^{R(\tau)}, & \pi'_R &\leftarrow g^{\alpha R(\tau)}, \\ \pi_O &\leftarrow g^{O(\tau)}, & \pi'_O &\leftarrow g^{\alpha O(\tau)}, \\ \pi_H &\leftarrow g^{H(\tau)}, & \pi'_H &\leftarrow g^{\alpha H(\tau)}.\end{aligned}$$

- ✓ Calculates the additional commitment π_β as follows:

$$\pi_\beta \leftarrow g^{\beta_L L(\tau)+\beta_R R(\tau)+\beta_O O(\tau)}$$

- ✓ Publishes $\boldsymbol{\pi} = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi'_H, \pi_\beta)$ as a proof.

Verify(vp, $\mathbf{x}$, $\boldsymbol{\pi}$)

Upon receiving $\boldsymbol{\pi} = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi'_H, \pi_\beta)$, the verifier $\mathcal{V}$ checks:

$$\begin{aligned}e(\pi_L, \pi_R) &= e(g^{Z(\tau)}, \pi_H) \cdot e(\pi_O, g), && \text{// Polynomial Equality Test} \\ e(\pi_L, g^\alpha) &= e(\pi'_L, g), \quad e(\pi_R, g^\alpha) = e(\pi'_R, g), && \text{// Proof of Exponent} \\ e(\pi_O, g^\alpha) &= e(\pi'_O, g), \quad e(\pi_H, g^\alpha) = e(\pi'_H, g), && \text{// Proof of Exponent} \\ e(\pi_L, g^{\gamma\beta_L}) \cdot e(\pi_R, g^{\gamma\beta_R}) \cdot e(\pi_O, g^{\gamma\beta_O}) &= e(\pi_\beta, g^\gamma). && \text{// Extended Witness Consistency}\end{aligned}$$

0.1.4 Attempt #4: Splitting the Extended Witness

Now, recall that the actual circuit C is defined for some statement $\mathbf{x}$ and witness $\mathbf{w}$. According to the Circuit Satisfability Problem, the prover $\mathcal{P}$ wants to convince the verifier $\mathcal{V}$ that he knows the witness $\mathbf{w}$ such that the circuit $C(\mathbf{x}, \mathbf{w}) = 0$. Up until now, we have been using the extended witness $\widetilde{\mathbf{w}}$ to represent the trace of computation. However, we can split the witness into two parts: the first part $\mathbf{w}_{\text{mid}}$ — intermediate witness that contains the private witness $R\mathbf{w}$ and intermediate variables values, and the second part $\mathbf{w}_{\text{io}}$ — input/output witness that contains public statement information (e.g., $\mathbf{x}$). Suppose for vector $\widetilde{\mathbf{w}} = (\widetilde{w}_1, \ldots, \widetilde{w}_n)$ we pick out the set of indeces $\mathcal{I}_{\text{mid}} \subset [n]$ to represent the intermediate witness and $\mathcal{I}_{\text{io}} \subset [n]$ to represent the input/output witness (of course, $\mathcal{I}_{\text{mid}} \cap \mathcal{I}_{\text{io}} = \emptyset$ and $\mathcal{I}_{\text{mid}} \cup \mathcal{I}_{\text{io}} = [n]$).

Now, how do we split the proofs $\pi_L, \pi_R, \ldots$ into two parts? Well, consider for instance π_L:

$$\pi_L = g^{\sum_{i=0}^{n} w_i L_i(\tau)}.$$

We split this expression as follows:

$$\pi_L = \underbrace{g^{\sum_{i \in \mathcal{I}_{\text{mid}}} w_i L_i(\tau)}}_{\text{new } \pi_L} \times \underbrace{g^{\sum_{i \in \mathcal{I}_{\text{io}}} w_i L_i(\tau)}}_{\pi_{L,\text{io}}}.$$

This way, the prover $\mathcal{P}$ first calculates the new commitment π_L using only the intermediate witness $\mathbf{w}_{\text{mid}}$ and then the verifier can compute the "public" portion of the proof $\pi_{L,\text{io}}$ using the input/output witness $\mathbf{w}_{\text{io}}$ (which is typically quite easy to do since $|\mathcal{I}_{\text{io}}|$ is typically much smaller than $|\mathcal{I}_{\text{mid}}|$). The same goes for other parts of the proof $\boldsymbol{\pi}$.

Now, let us formulate the updated SNARK protocol with the extended witness split.

Attempt #4: Sound SNARK for Public/Private Inputs

Suppose we are given a circuit C with a maximum degree d of polynomials used underneath. Thus, all parties additionally know the target polynomial $Z(x)$. Additionally, we know that $\mathcal{I}_{\text{io}}$ corresponds to public signals, while $\mathcal{I}_{\text{mid}}$ corresponds to private signals.

Setup(1^λ)

The *trusted party* conducts the following steps:

- ✓ Picks random values $\tau, \alpha, \beta_L, \beta_R, \beta_O, \gamma \xleftarrow{R} \mathbb{F}$.
- ✓ **Outputs** prover parameters pp and verification parameters vp:

$$\begin{aligned}
\mathsf{pp} \leftarrow \big\{ &\{g^{\tau^i}\}_{i\in[d]}, \{g^{L_i(\tau)}, g^{\alpha L_i(\tau)}, g^{R_i(\tau)}, g^{\alpha R_i(\tau)}, \\
&g^{O_i(\tau)}, g^{\alpha O_i(\tau)}, g^{\beta_L L_i(\tau)+\beta_R R_i(\tau)+\beta_O O_i(\tau)}\}_{i\in\mathcal{I}_{\text{mid}}}\big\} \\
\mathsf{vp} \leftarrow \big\{ &g^{Z(\tau)}, g^{\alpha}, g^{\beta_L}, g^{\beta_R}, g^{\beta_O}, g^{\beta_L\gamma}, g^{\beta_R\gamma}, g^{\beta_O\gamma}, g^{\gamma}, \{g^{L_i(\tau)}, g^{R_i(\tau)}, g^{O_i(\tau)}\}_{i\in\mathcal{I}_{\text{io}}}\big\}
\end{aligned}$$

- ✓ **Deletes** aforementioned random scalars (toxic waste).

Prove(pp, $\mathbf{x}$, $\mathbf{w}$)

The prover $\mathcal{P}$ conducts the following steps:

- ✓ Runs the circuit to get $\mathbf{w}$ and $L(x), R(x), O(x)$.
- ✓ Calculates $H(x) \leftarrow (L(x)R(x) - O(x))/Z(x)$.
- ✓ Splits $L(x) = L_{\text{mid}}(x) + L_{\text{io}}(x)$ — intermediate and input/output parts. That being said, $L_{\text{mid}}(x) = \sum_{i\in\mathcal{I}_{\text{mid}}} L_i(x)$. Repeat for $R(x)$ and $O(x)$.
- ✓ Calculates the following values:

$$\begin{aligned}
\pi_L &\leftarrow g^{L_{\text{mid}}(\tau)}, & \pi'_L &\leftarrow g^{\alpha L_{\text{mid}}(\tau)}, \\
\pi_R &\leftarrow g^{R_{\text{mid}}(\tau)}, & \pi'_R &\leftarrow g^{\alpha R_{\text{mid}}(\tau)}, \\
\pi_O &\leftarrow g^{O_{\text{mid}}(\tau)}, & \pi'_O &\leftarrow g^{\alpha O_{\text{mid}}(\tau)}, \\
\pi_H &\leftarrow g^{H(\tau)}, & \pi'_H &\leftarrow g^{\alpha H(\tau)}, \\
\pi_\beta &\leftarrow g^{\beta_L L(\tau)+\beta_R R(\tau)+\beta_O O(\tau)}.
\end{aligned}$$

- ✓ Publishes $\boldsymbol{\pi} = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi'_H, \pi_\beta)$ as a proof.

Verify(vp, $\mathbf{x}$, $\boldsymbol{\pi}$)

Upon receiving $\boldsymbol{\pi} = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi'_H, \pi_\beta)$, the verifier $\mathcal{V}$:

- ✓ Finds $\pi^*_L \leftarrow \pi_L \prod_{i\in\mathcal{I}_{\text{io}}} (g^{L_i(\tau)})^{w_i}$, $\pi^*_R \leftarrow \pi_R \prod_{i\in\mathcal{I}_{\text{io}}} (g^{R_i(\tau)})^{w_i}$, $\pi^*_O \leftarrow \pi_O \prod_{i\in\mathcal{I}_{\text{io}}} (g^{O_i(\tau)})^{w_i}$
- ✓ Checks whether all of the following conditions hold:

$$\begin{aligned}
e(\pi^*_L, \pi^*_R) &= e(g^{Z(\tau)}, \pi_H) \cdot e(\pi^*_O, g), && \text{// Polynomial Equality Test} \\
e(\pi_L, g^\alpha) = e(\pi'_L, g), \quad e(\pi_R, g^\alpha) &= e(\pi'_R, g), && \text{// Proof of Exponent} \\
e(\pi_O, g^\alpha) = e(\pi'_O, g), \quad e(\pi_H, g^\alpha) &= e(\pi'_H, g), && \text{// Proof of Exponent} \\
e(\pi_L, g^{\gamma\beta_L}) \cdot e(\pi_R, g^{\gamma\beta_R}) \cdot e(\pi_O, g^{\gamma\beta_O}) &= e(\pi_\beta, g^\gamma). && \text{// Extended Witness Consistency}
\end{aligned}$$

0.1.5 Attempt #5: Making SNARK Zero-Knowledge

Finally, we came to the point where we need to make the protocol zero-knowledge. As it turns out, it is not that hard to do (compared to what we have done so far)!

Remark. Currently, you might have a reasonable question: the proof contains quantities such as $g^{L(\tau)}$, $g^{R(\tau)}$, $g^{O(\tau)}$ and variations of them. As long as discrete logarithm holds, there is no PPT adversary that would be able to extract coefficients of $L(x)$, $R(x)$, $O(x)$ from published KZG commitments (and respective PoE shifts and witness consistency proof). So what could be the issue?
This reasoning is correct. In other words, the adversary will not learn coefficients of polynomials and therefore will not be able to get the witness fully. However, the zero-knowledge property ensures that the adversary cannot draw conclusions based on proof $\boldsymbol{\pi}$. However, in our current version, we do have SNARK, but not zk-SNARK. For instance, currently anyone can check whether $L \equiv R$ (by checking $\pi_L = \pi_R$) or that $L \equiv 11R$ (by checking $\pi_L = \pi_R^{11}$).

The main idea to make our protocol zero-knowledge is to "shift" our commitments by some random factor δ. Of course, this "shift" δ must be different for each commitment and must be chosen by the prover $\mathcal{P}$. This way, we propose to modify the commitments as follows:

$$\pi_L = g^{\sum_{i\in\mathcal{I}_{\text{mid}}} w_i L_i(\tau) + \delta_L Z(\tau)} = (g^{Z(\tau)})^{\delta_L} \prod_{i\in\mathcal{I}_{\text{mid}}} (g^{L_i(\tau)})^{w_i}, \qquad \pi'_L = (g^{\alpha Z(\tau)})^{\delta_L} \prod_{i\in\mathcal{I}_{\text{mid}}} (g^{\alpha L_i(\tau)})^{w_i}$$

$$\pi_R = g^{\sum_{i\in\mathcal{I}_{\text{mid}}} w_i R_i(\tau) + \delta_R Z(\tau)} = (g^{Z(\tau)})^{\delta_R} \prod_{i\in\mathcal{I}_{\text{mid}}} (g^{R_i(\tau)})^{w_i}, \qquad \pi'_R = (g^{\alpha Z(\tau)})^{\delta_R} \prod_{i\in\mathcal{I}_{\text{mid}}} (g^{\alpha R_i(\tau)})^{w_i}$$

$$\pi_O = g^{\sum_{i\in\mathcal{I}_{\text{mid}}} w_i O_i(\tau) + \delta_O Z(\tau)} = (g^{Z(\tau)})^{\delta_O} \prod_{i\in\mathcal{I}_{\text{mid}}} (g^{O_i(\tau)})^{w_i}, \qquad \pi'_O = (g^{\alpha Z(\tau)})^{\delta_O} \prod_{i\in\mathcal{I}_{\text{mid}}} (g^{\alpha O_i(\tau)})^{w_i}$$

for randomly selected $\delta_L, \delta_R, \delta_O \xleftarrow{R} \mathbb{Z}_q$. Good, we concealed all the information about the witness $\mathbf{w}$ in the commitment. However, we need to make sure that the verifier $\mathcal{V}$ can still verify the proof, but without modifying the verification mechanism itself. Notice that PoE verifications are still valid, but we need to modify something to make polynomial equality test hold. This can be done by perturbating the polynomial $H(x)$ by some (currently) unknown value Δ_H. Let us derive it from the polynomial equality test:

$$(L(x) + \delta_L Z(x))(R(x) + \delta_R Z(x)) = (H(x) + \Delta_H)Z(x) + (O(x) + \delta_O Z(x)),$$

which, by expanding, gives us the following equation:

$$\cancel{L(x)R(x)} + \delta_R L(x)Z(x) + \delta_L Z(x)R(x) + \delta_L\delta_R Z(x)^2 = \cancel{H(x)Z(x) + O(x)} + \Delta_H Z(x) + \delta_O Z(x),$$

where we can cancel out $L(x)R(x)$ and $H(x)Z(x) + O(x)$ terms since they are equal based on initial construction. This way, we get the following expression for Δ_H:

$$\Delta_H = \delta_O + \delta_R L(x) + \delta_L R(x) + \delta_L\delta_R Z(x)$$

Therefore, our fourth proof system becomes:

$$\pi_H = g^{H(\tau) + \delta_O + \delta_R L(\tau) + \delta_L R(\tau) + \delta_L\delta_R Z(\tau)}, \qquad \pi'_H = g^{\alpha(H(\tau) + \delta_O + \delta_R L(\tau) + \delta_L R(\tau) + \delta_L\delta_R Z(\tau))}.$$

Finally, we need to make sure that our witness consistency proof π_β is still valid. Since previously we had $\pi_\beta = g^{\beta_L L(\tau)+\beta_R R(\tau)+\beta_O O(\tau)}$, we need to modify it to include the new δ values. Namely, we change $L(\tau)$ to $L(\tau)+\delta_L Z(\tau)$, $R(\tau)$ to $R(\tau)+\delta_R Z(\tau)$, and $O(\tau)$ to $O(\tau)+\delta_O Z(\tau)$. This way, our new π_β becomes:

This can be done by changing it into:

$$\begin{aligned}\pi_\beta &= g^{\beta_L L(\tau)+\beta_R R(\tau)+\beta_O O(\tau)+(\delta_L\beta_L+\delta_R\beta_R+\delta_O\beta_O)Z(\tau)}\\ &= \left(g^{\beta_L Z(\tau)}\right)^{\delta_L}\left(g^{\beta_R Z(\tau)}\right)^{\delta_R}\left(g^{\beta_O Z(\tau)}\right)^{\delta_O} g^{\beta_L L(\tau)+\beta_R R(\tau)+\beta_O O(\tau)}.\end{aligned}$$

Oof. Finally, we also have a zero-knowledge property for our SNARK. Let us summarize what has changed for the prover $\mathcal{P}$.

Proposition 0.1.2 (Including ZK in general-purpose SNARK)**.** Now, the prover $\mathcal{P}$ samples random scalars $\delta_L, \delta_R, \delta_O \xleftarrow{R} \mathbb{Z}_q$ and calculates the following commitments:

- **Updated Commitments for L, R, O:** Now, the prover $\mathcal{P}$ needs to calculate the commitments π_L, π_R, π_O with the additional δ's values to ensure zero-knowledge property:

$$\begin{aligned}\pi_L &\leftarrow g^{\sum_{i\in\mathcal{I}_{\text{mid}}} w_i L_i(\tau)+\delta_L Z(\tau)}, & \pi'_L &\leftarrow g^{\alpha\sum_{i\in\mathcal{I}_{\text{mid}}} w_i L_i(\tau)+\delta_L Z(\tau)},\\ \pi_R &\leftarrow g^{\sum_{i\in\mathcal{I}_{\text{mid}}} w_i R_i(\tau)+\delta_R Z(\tau)}, & \pi'_R &\leftarrow g^{\alpha\sum_{i\in\mathcal{I}_{\text{mid}}} w_i R_i(\tau)+\delta_R Z(\tau)},\\ \pi_O &\leftarrow g^{\sum_{i\in\mathcal{I}_{\text{mid}}} w_i O_i(\tau)+\delta_O Z(\tau)}, & \pi'_O &\leftarrow g^{\alpha\sum_{i\in\mathcal{I}_{\text{mid}}} w_i O_i(\tau)+\delta_O Z(\tau)}.\end{aligned}$$

- **Updated Commitment for H:** The prover $\mathcal{P}$ needs to calculate the commitment π_H with the additional δ's values to ensure polynomial equality test is satisfied:

$$\pi_H \leftarrow g^{H(\tau)+\delta_O+\delta_R L(\tau)+\delta_L R(\tau)+\delta_L\delta_R Z(\tau)}, \quad \pi'_H \leftarrow g^{\alpha(H(\tau)+\delta_O+\delta_R L(\tau)+\delta_L R(\tau)+\delta_L\delta_R Z(\tau))}.$$

- **Updated Witness Consistency Proof:** The prover $\mathcal{P}$ needs to calculate the commitment π_β with the additional δ's values to ensure witness consistency:

$$\pi_\beta \leftarrow \left(g^{\beta_L Z(\tau)}\right)^{\delta_L}\left(g^{\beta_R Z(\tau)}\right)^{\delta_R}\left(g^{\beta_O Z(\tau)}\right)^{\delta_O} g^{\beta_L L(\tau)+\beta_R R(\tau)+\beta_O O(\tau)}.$$

Let us now look at the final version of our basic SNARK protocol.

Attempt #5: Turning SNARK into zk-SNARK

Suppose we are given a circuit C with a maximum degree d of polynomials used underneath. Thus, all parties additionally know the target polynomial $Z(x)$. Additionally, we know that $\mathcal{I}_{\text{io}}$ corresponds to public signals, while $\mathcal{I}_{\text{mid}}$ corresponds to private signals.

Setup(1^λ)

The *trusted party* conducts the following steps:

- ✓ Picks random values $\tau, \alpha, \beta_L, \beta_R, \beta_O, \gamma \xleftarrow{R} \mathbb{F}$.
- ✓ **Outputs** prover parameters pp and verification parameters vp:

$$\begin{aligned}
\mathsf{pp} \leftarrow \{ & \{g^{\tau^i}\}_{i\in[d]}, \{g^{Z(\tau)}, g^{L_i(\tau)}, g^{\alpha L_i(\tau)}, g^{R_i(\tau)}, g^{\alpha R_i(\tau)}, \\
& g^{O_i(\tau)}, g^{\alpha O_i(\tau)}, g^{\beta_L L_i(\tau)}, g^{\beta_R R_i(\tau)}, g^{\beta_O O_i(\tau)}\}_{i\in\mathcal{I}_{\text{mid}}}\} \\
\mathsf{vp} \leftarrow \{ & g^{Z(\tau)}, g^{\alpha}, g^{\beta_L}, g^{\beta_R}, g^{\beta_O}, g^{\beta_L\gamma}, g^{\beta_R\gamma}, g^{\beta_O\gamma}, g^{\gamma}, \{g^{L_i(\tau)}, g^{R_i(\tau)}, g^{O_i(\tau)}\}_{i\in\mathcal{I}_{\text{io}}}\}
\end{aligned}$$

- ✓ **Deletes** aforementioned random scalars (toxic waste).

Prove(pp, $\mathbf{x}$, $\mathbf{w}$)

The prover $\mathcal{P}$ conducts the following steps:

- ✓ Runs the circuit to get $\mathbf{w}$ and $L(x), R(x), O(x)$.
- ✓ Calculates $H(x) \leftarrow (L(x)R(x) - O(x))/Z(x)$.
- ✓ Splits $L(x) = L_{\text{mid}}(x) + L_{\text{io}}(x)$ — intermediate and input/output parts. That being said, $L_{\text{mid}}(x) = \sum_{i\in\mathcal{I}_{\text{mid}}} L_i(x)$. Repeat for $R(x)$ and $O(x)$.
- ✓ Samples $\delta_L, \delta_R, \delta_O \xleftarrow{R} \mathbb{F}$ and calculates the following values:

$$\begin{aligned}
\pi_L &\leftarrow g^{L_{\text{mid}}(\tau)}\left(g^{Z(\tau)}\right)^{\delta_L}, \quad \pi'_L \leftarrow g^{\alpha L_{\text{mid}}(\tau)}\left(g^{\alpha Z(\tau)}\right)^{\delta_L}, \\
\pi_R &\leftarrow g^{R_{\text{mid}}(\tau)}\left(g^{Z(\tau)}\right)^{\delta_R}, \quad \pi'_R \leftarrow g^{\alpha R_{\text{mid}}(\tau)}\left(g^{Z(\tau)}\right)^{\delta_R}, \\
\pi_O &\leftarrow g^{O_{\text{mid}}(\tau)}\left(g^{Z(\tau)}\right)^{\delta_O}, \quad \pi'_O \leftarrow g^{\alpha O_{\text{mid}}(\tau)}\left(g^{Z(\tau)}\right)^{\delta_O}, \\
\pi_H &\leftarrow g^{H(\tau)}(g^{\delta_O})(g^{R(\tau)})^{\delta_L}(g^{L(\tau)})^{\delta_R}(g^{Z(\tau)})^{\delta_L\delta_R}, \\
\pi'_H &\leftarrow g^{\alpha H(\tau)}(g^{\delta_O})(g^{R(\tau)})^{\delta_L}(g^{L(\tau)})^{\delta_R}(g^{Z(\tau)})^{\delta_L\delta_R}, \\
\pi_\beta &\leftarrow \left(g^{\beta_L Z(\tau)}\right)^{\delta_L}\left(g^{\beta_R Z(\tau)}\right)^{\delta_R}\left(g^{\beta_O Z(\tau)}\right)^{\delta_O} g^{\beta_L L(\tau)+\beta_R R(\tau)+\beta_O O(\tau)}.
\end{aligned}$$

- ✓ Publishes $\boldsymbol{\pi} = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi'_H, \pi_\beta)$ as a proof.

Verify(vp, $\mathbf{x}$, $\boldsymbol{\pi}$)

Upon receiving $\boldsymbol{\pi} = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi'_H, \pi_\beta)$, the verifier $\mathcal{V}$:

- ✓ Finds $\pi^*_L \leftarrow \pi_L \prod_{i\in\mathcal{I}_{\text{io}}}(g^{L_i(\tau)})^{w_i}$, $\pi^*_R \leftarrow \pi_R \prod_{i\in\mathcal{I}_{\text{io}}}(g^{R_i(\tau)})^{w_i}$, $\pi^*_O \leftarrow \pi_O \prod_{i\in\mathcal{I}_{\text{io}}}(g^{O_i(\tau)})^{w_i}$
- ✓ Checks whether all of the following conditions hold:

$$\begin{aligned}
e(\pi^*_L, \pi^*_R) &= e(g^{Z(\tau)}, \pi_H)\cdot e(\pi^*_O, g), && \textit{// Polynomial Equality Test} \\
e(\pi_L, g^\alpha) = e(\pi'_L, g), \quad e(\pi_R, g^\alpha) &= e(\pi'_R, g), && \textit{// Proof of Exponent} \\
e(\pi_O, g^\alpha) = e(\pi'_O, g), \quad e(\pi_H, g^\alpha) &= e(\pi'_H, g), && \textit{// Proof of Exponent} \\
e(\pi_L, g^{\gamma\beta_L})\cdot e(\pi_R, g^{\gamma\beta_R})\cdot e(\pi_O, g^{\gamma\beta_O}) &= e(\pi_\beta, g^\gamma). && \textit{// Extended Witness Consistency}
\end{aligned}$$

0.2 Real Protocols

While our built protocol is zk-SNARK, it is still not optimized for practical use. In fact, let us recap what is complexity of our protocol.

Proposition 0.2.1 (Complexity of the Basic Protocol)**.** Suppose circuit consists of n gates. Then, the complexity of the basic protocol is as follows:

- **Proof Size:** $O(1)$ — we have a constant number of group elements.
- **Setup Time:** $O(n)$ — we need to calculate powers of τ and evaluations at τ.
- **Prover Time:** $O(n \log n)$ — using FFT and wise choice of Ω.
- **Verifier Time:** $O(1)$ — we have a constant number of pairings to evaluate.

However, $O(1)$ is not very descriptive for proof and verifier complexities, so let us provide a more detailed analysis.

- **Proof Size:** 9 $\mathbb{G}$ group elements.
- **Verifier Time:** 15 pairings and $O(|\mathcal{I}_{\text{io}}|)$ group multiplications.

Now, this is not bad at all! In fact, this is already practical for many applications. However, we can do better by a more clever choice of constants and terms. This is exactly what is done by Bryan Parno and Craig Gentry in their research "Pinocchio: Nearly Practical Verifiable Computation".

0.2.1 Pinocchio Protocol

We first begin from the non-zero-knowledge version of the Pinocchio Protocol and then extend it to the zero-knowledge version.

Setup Procedure. The Pinocchio Protocol exploits the idea that we might choose different generators for $L(x)$, $R(x)$, and $O(x)$. Namely, the protocol begins from choosing random $\rho_L, \rho_R \xleftarrow{R} \mathbb{F}$. Then, we define $\rho_O \triangleq \rho_L \rho_R$, and finally introduce the following generators:

$$g_L \triangleq g^{\rho_L}, \quad g_R \triangleq g^{\rho_R}, \quad g_O \triangleq g^{\rho_O} = g^{\rho_L \rho_R}$$

The next change is that we use different α values for $L(x)$, $R(x)$, and $O(x)$. Namely, we define $\alpha_L, \alpha_R, \alpha_O \xleftarrow{R} \mathbb{F}$ and use them in the commitments. This way, in the setup phase, we additionally prepare the following values:

$$\{g_L^{L_i(\tau)}, g_L^{\alpha_L L_i(\tau)}, g_R^{R_i(\tau)}, g_R^{\alpha_R R_i(\tau)}, g_O^{O_i(\tau)}, g_O^{\alpha_O O_i(\tau)}\}_{i \in \mathcal{I}_{\text{mid}}}$$

Instead of using three different β's, the Pinocchio Protocol uses a single $\beta \xleftarrow{R} \mathbb{F}$:

$$\{g_L^{\beta L_i(\tau)}, g_R^{\beta R_i(\tau)}, g_O^{\beta O_i(\tau)}\}_{i \in \mathcal{I}_{\text{mid}}}$$

And finally, since we also have a $\gamma \xleftarrow{R} \mathbb{F}$ for the extended witness consistency and some other missing quantities, we also prepare the following values:

$$g_O^{Z(\tau)}, g^{\alpha_L}, g^{\alpha_R}, g^{\alpha_O}, g^{\beta\gamma}, g^{\gamma}$$

Proving. Again, suppose intermediate polynomials for indices $\mathcal{I}_{\text{mid}}$ are $L_{\text{mid}}(x) = \sum_{i \in \mathcal{I}_{\text{mid}}} w_i L_i(x)$ and similarly for $R_{\text{mid}}(x)$ and $O_{\text{mid}}(x)$. The prover $\mathcal{P}$ calculates the following commitments:

$$\pi_L \leftarrow g_L^{L_{\text{mid}}(\tau)}, \quad \pi'_L \leftarrow g_L^{\alpha_L L_{\text{mid}}(\tau)},$$
$$\pi_R \leftarrow g_R^{R_{\text{mid}}(\tau)}, \quad \pi'_R \leftarrow g_R^{\alpha_R R_{\text{mid}}(\tau)},$$
$$\pi_O \leftarrow g_O^{O_{\text{mid}}(\tau)}, \quad \pi'_O \leftarrow g_O^{\alpha_O O_{\text{mid}}(\tau)},$$
$$\pi_H \leftarrow g^{H(\tau)}, \quad \pi_\beta \leftarrow g_L^{\beta L_{\text{mid}}(\tau)} g_R^{\beta R_{\text{mid}}(\tau)} g_O^{\beta O_{\text{mid}}(\tau)},$$

and then uses these commitments in the proof $\boldsymbol{\pi} = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi_\beta)$.

Verification. First, the verifier checks the PoE properties:

$$e(\pi_L, g_L^{\alpha_L}) = e(\pi'_L, g_L), \quad e(\pi_R, g_R^{\alpha_R}) = e(\pi'_R, g_R), \quad e(\pi_O, g_O^{\alpha_O}) = e(\pi'_O, g_O).$$

Next, we check the extended witness consistency:

$$e(\pi_L \pi_R \pi_O, g^{\beta\gamma}) = e(\pi_\beta, g^\gamma)$$

The verifier restores the full proof $\pi_L^*, \pi_R^*, \pi_O^*$ by multiplying the public part of the proof with the corresponding private part. This is done as follows:

$$\pi_L^* \leftarrow \pi_L \prod_{i \in \mathcal{I}_{\text{io}}} (g_L^{L_i(\tau)})^{w_i}, \quad \pi_R^* \leftarrow \pi_R \prod_{i \in \mathcal{I}_{\text{io}}} (g_R^{R_i(\tau)})^{w_i}, \quad \pi_O^* \leftarrow \pi_O \prod_{i \in \mathcal{I}_{\text{io}}} (g_O^{O_i(\tau)})^{w_i}.$$

And finally, we check the polynomial equality test:

$$e(\pi_L^*, \pi_R^*) = e(g_O^{Z(\tau)}, \pi_H) \cdot e(\pi_O^*, g)$$

Zero-Knowledge Extension. The zero-knowledge extension of the Pinocchio Protocol is done exactly in the same way as we did for the basic SNARK protocol. Namely, we introduce random scalars $\delta_L, \delta_R, \delta_O \xleftarrow{R} \mathbb{F}$ and calculate the following commitments:

$$\pi_L \leftarrow g_L^{L_{\text{mid}}(\tau)} \left(g_L^{Z(\tau)}\right)^{\delta_L}, \quad \pi'_L \leftarrow g_L^{\alpha_L L_{\text{mid}}(\tau)} \left(g_L^{\alpha_L Z(\tau)}\right)^{\delta_L},$$
$$\pi_R \leftarrow g_R^{R_{\text{mid}}(\tau)} \left(g_R^{Z(\tau)}\right)^{\delta_R}, \quad \pi'_R \leftarrow g_R^{\alpha_R R_{\text{mid}}(\tau)} \left(g_R^{\alpha_R Z(\tau)}\right)^{\delta_R},$$
$$\pi_O \leftarrow g_O^{O_{\text{mid}}(\tau)} \left(g_O^{Z(\tau)}\right)^{\delta_O}, \quad \pi'_O \leftarrow g_O^{\alpha_O O_{\text{mid}}(\tau)} \left(g_O^{\alpha_O Z(\tau)}\right)^{\delta_O},$$
$$\pi_H \leftarrow g^{H(\tau)} (g^{\delta_O})(g^{\delta_L})(g^{\delta_R})(g^{\delta_L \delta_R}),$$
$$\pi_\beta \leftarrow \left(g_L^{\beta Z(\tau)}\right)^{\delta_L} \left(g_R^{\beta Z(\tau)}\right)^{\delta_R} \left(g_O^{\beta Z(\tau)}\right)^{\delta_O} g_L^{\beta L_{\text{mid}}(\tau)} g_R^{\beta R_{\text{mid}}(\tau)} g_O^{\beta O_{\text{mid}}(\tau)}$$

Let us see what has changed in the Pinocchio Protocol.

Proposition 0.2.2. Pinocchio Protocol Proof size and verifier time complexity, compared to the basic SNARK protocol, are as follows:

- **Proof Size: 8** $\mathbb{G}$ group elements.
- **Verifier Time: 11** pairings and $O(|\mathcal{I}_{\text{io}}|)$ group multiplications (**4 less!**)

Let us see the complete specification of the Pinocchio Protocol.

Pinocchio Protocol

Suppose we are given a circuit C with a maximum degree d of polynomials used underneath. Thus, all parties additionally know the target polynomial $Z(x)$. Additionally, we know that $\mathcal{I}_{\text{io}}$ corresponds to public signals, while $\mathcal{I}_{\text{mid}}$ corresponds to private signals.

Setup(1^λ)

The *trusted party* conducts the following steps:

- ✓ Picks random values $\tau, \alpha_L, \alpha_R, \alpha_O, \beta, \gamma, \rho_L, \rho_R \xleftarrow{R} \mathbb{F}$.
- ✓ Calculates $\rho_O \leftarrow \rho_L \rho_R$ and defines generators $g_L \leftarrow g^{\rho_L}, g_R \leftarrow g^{\rho_R}, g_O \leftarrow g^{\rho_O}$.
- ✓ **Outputs** prover parameters pp and verification parameters vp:

$$\begin{aligned}
\mathsf{pp} \leftarrow \big\{ &\{g^{\tau^i}\}_{i\in[d]}, \{g_L^{L_i(\tau)}, g_L^{\alpha_L L_i(\tau)}, g_R^{R_i(\tau)}, g_R^{\alpha_R R_i(\tau)}, g_O^{O_i(\tau)}, g_O^{\alpha_O O_i(\tau)}, \\
&g_L^{\beta L_i(\tau)}, g_R^{\beta R_i(\tau)}, g_O^{\beta O_i(\tau)}\}_{i\in\mathcal{I}_{\text{mid}}}, \\
&g_L^{\alpha_L Z(\tau)}, g_L^{Z(\tau)}, g_R^{Z(\tau)}, g_O^{Z(\tau)}, g_R^{\alpha_R Z(\tau)}, g_O^{\alpha_O Z(\tau)}, g_L^{\beta Z(\tau)}, g_R^{\beta Z(\tau)}, g_O^{\beta Z(\tau)}\big\} \\
\mathsf{vp} \leftarrow \big\{ &g_O^{Z(\tau)}, g^{\alpha_L}, g^{\alpha_R}, g^{\alpha_O}, g^{\beta\gamma}, g^{\gamma}, \{g_L^{L_i(\tau)}, g_R^{R_i(\tau)}, g_O^{O_i(\tau)}\}_{i\in\mathcal{I}_{\text{io}}}\big\}
\end{aligned}$$

- ✓ **Deletes** aforementioned random scalars (toxic waste).

Prove(pp, $\mathbf{x}$, $\mathbf{w}$)

The prover $\mathcal{P}$ conducts the following steps:

- ✓ Runs the circuit to get $\mathbf{w}$ and $L(x), R(x), O(x)$.
- ✓ Calculates $H(x) \leftarrow (L(x)R(x) - O(x))/Z(x)$.
- ✓ Splits $L(x) = L_{\text{mid}}(x) + L_{\text{io}}(x)$ — intermediate and input/output parts. Repeat for $R(x)$ and $O(x)$.
- ✓ Samples $\delta_L, \delta_R, \delta_O \xleftarrow{R} \mathbb{F}$ and publishes the following values as a proof $\boldsymbol{\pi}$:

$$\begin{aligned}
\pi_L &\leftarrow g_L^{L_{\text{mid}}(\tau)} \left(g_L^{Z(\tau)}\right)^{\delta_L}, \quad \pi'_L \leftarrow g_L^{\alpha_L L_{\text{mid}}(\tau)} \left(g_L^{\alpha_L Z(\tau)}\right)^{\delta_L}, \\
\pi_R &\leftarrow g_R^{R_{\text{mid}}(\tau)} \left(g_R^{Z(\tau)}\right)^{\delta_R}, \quad \pi'_R \leftarrow g_R^{\alpha_R R_{\text{mid}}(\tau)} \left(g_R^{\alpha_R Z(\tau)}\right)^{\delta_R}, \\
\pi_O &\leftarrow g_O^{O_{\text{mid}}(\tau)} \left(g_O^{Z(\tau)}\right)^{\delta_O}, \quad \pi'_O \leftarrow g_O^{\alpha_O O_{\text{mid}}(\tau)} \left(g_O^{\alpha_O Z(\tau)}\right)^{\delta_O}, \\
\pi_H &\leftarrow g^{H(\tau)} (g^{\delta_O})(g^{\delta_L})(g^{\delta_R})(g^{\delta_L\delta_R}), \\
\pi_\beta &\leftarrow \left(g_L^{\beta Z(\tau)}\right)^{\delta_L} \left(g_R^{\beta Z(\tau)}\right)^{\delta_R} \left(g_O^{\beta Z(\tau)}\right)^{\delta_O} g_L^{\beta L_{\text{mid}}(\tau)} g_R^{\beta R_{\text{mid}}(\tau)} g_O^{\beta O_{\text{mid}}(\tau)}
\end{aligned}$$

Verify(vp, $\mathbf{x}$, $\boldsymbol{\pi}$)

Upon receiving $\boldsymbol{\pi} = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi_\beta)$, the verifier $\mathcal{V}$:

- ✓ Finds $\pi_L^* \leftarrow \pi_L \prod_{i\in\mathcal{I}_{\text{io}}} (g_L^{L_i(\tau)})^{w_i}, \pi_R^* \leftarrow \pi_R \prod_{i\in\mathcal{I}_{\text{io}}} (g_R^{R_i(\tau)})^{w_i}, \pi_O^* \leftarrow \pi_O \prod_{i\in\mathcal{I}_{\text{io}}} (g_O^{O_i(\tau)})^{w_i}$
- ✓ Checks whether all of the following conditions hold:

$$\begin{aligned}
e(\pi_L^*, \pi_R^*) &= e(g_O^{Z(\tau)}, \pi_H) \cdot e(\pi_O^*, g), && \text{// Polynomial Equality Test} \\
e(\pi_L, g_L^{\alpha_L}) = e(\pi'_L, g_L), \quad e(\pi_R, g_R^{\alpha_R}) &= e(\pi'_R, g_R), && \text{// Proof of Exponent} \\
e(\pi_O, g_O^{\alpha_O}) &= e(\pi'_O, g_O), && \text{// Proof of Exponent} \\
e(\pi_L \pi_R \pi_O, g^{\gamma\beta}) &= e(\pi_\beta, g^\gamma). && \text{// Extended Witness Consistency}
\end{aligned}$$

0.2.2 Groth16 Protocol

Finally, Groth16 allows to reduce the number of pairings **down to 3**! This is done through a technique called **Generic Group Model** (GGM for short). Simply put, GGM allows the adversary to only make oracle requests to compute the group operations. For example, having a set $\{g^{\alpha R_i(\tau)}\}_{i\in[d]}$, adversary can compute only linear combinations of these values. In the particular case of Groth16, instead of considering $L_i(x)$, $R_i(x)$, and $O_i(x)$ separately, we construct their linear combinations as $Q_i(x) := \beta L_i(x) + \alpha R_i(x) + O_i(x)$, where α and β are toxic parameters.

Let us now concretely describe the Groth16 construction.

Asymmetric Pairing. One change which was made in Groth16 is using the asymmetric pairing function $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ over groups $\mathbb{G}_1$ and $\mathbb{G}_2$ (for more details, see **??**). Suppose respective group generators are $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$. The primary reason for this is that calculating the pairing in asymmetric setting is more efficient than in symmetric setting. Other than that, Groth16 can be easily formulated using symmetric pairings.

Setup Procedure. In Groth16, we need only five random scalars: $\alpha, \beta, \gamma, \delta, \tau \xleftarrow{R} \mathbb{F}$. Now, the prover key pk looks as follows:

$$\mathsf{pp} \leftarrow \left(g_1^{\alpha}, g_1^{\beta}, g_1^{\delta}, \left\{ g_1^{\tau^i}, \frac{\beta L_i(\tau) + \alpha R_i(\tau) + O_i(\tau)}{\gamma}, \frac{\tau^i Z(\tau)}{\delta} \right\}_{i\in[n]}, g_2^{\beta}, g_2^{\delta}, g_2^{\gamma}, \{g_2^{\tau^i}\}_{i\in[d]} \right)$$

Proving. Sample random $\delta_L, \delta_R \xleftarrow{R} \mathbb{F}$ and compute the following values:

$$\pi_L \leftarrow g_1^{\alpha + \sum_{i=1}^{n} w_i L_i(\tau) + \delta_L \delta}, \quad \pi_R \leftarrow g_2^{\beta + \sum_{i=0}^{n} w_i R_i(\tau) + \delta_R \delta}, \quad \pi_O \leftarrow g_1^{\frac{Q_{\mathsf{mid}}(\tau) + H(\tau) Z(\tau)}{\delta} + L\delta_R + R\delta_L - \delta_L \delta_R \delta},$$

where we denoted $Q_{\mathsf{mid}} := \sum_{i\in\mathcal{I}_{\mathsf{mid}}} w_i(\beta L_i(\tau) + \alpha R_i(\tau) + O_i(\tau))$.

Verification. The verifier first calculates the following value:

$$\pi_{\mathsf{io}} \leftarrow g_1^{\sum_{i\in\mathcal{I}_{\mathsf{io}}} w_i(\beta L_i(\tau) + \alpha R_i(\tau) + O_i(\tau))/\gamma},$$

and then checks the following single condition:

$$e(\pi_L, \pi_R) = e(g_1^{\alpha}, g_2^{\beta}) e(\pi_{\mathsf{io}}, g_2^{\gamma}) e(\pi_O, g_2^{\delta})$$

Note that $e(g_1^{\alpha}, g_2^{\beta})$ can be additionally hard-coded in the verifier, thus reducing the number of pairings to 3. Finally, the proof's size is now reduced to 3 group elements: two from $\mathbb{G}_1$, and one from $\mathbb{G}_2$.